REPORT · 2026年8月

造像类文物三维建模自动化系统
实机测试操作指南

测试环境准备 · 模块操作流程 · 预期结果与判定 · 问题应对 · 后续开发路线图

文保数字化研发组
2026年8月

目 录

第一章 测试概述 4

第二章 测试环境准备 5

2.1 硬件与软件要求 5

2.2 环境检查步骤 5

2.3 测试数据准备 5

2.4 配置文件准备 5

第三章 各功能模块操作流程 7

3.1 配置校验（validate） 7

3.2 队列预览（batch --dry-run） 7

3.3 批量处理（batch，核心功能） 8

3.4 报表重生成（report） 8

3.5 交付包核验（M6 产物） 8

3.6 可选模块 X1 M3C2 多时相（独立调用） 8

3.7 可选模块 X2 CIEDE2000 色差（独立调用） 8

第四章 预期结果与判定标准 10

4.1 逐命令判定表 10

4.2 质量门判定标准（M5） 10

4.3 失败隔离判定 10

第五章 常见问题与应对建议 11

第六章 后续持续开发方向 12

6.1 后续可迭代的功能与优化方向 12

6.2 面向未来的发展目标与扩展路线 12

6.3 GUI 开发方案与界面设计思路 12

6.4 完整的软件化改造路径 13

6.5 完备 GUI 窗口的架构设计 13

6.6 软件图标化、品牌化与发布打包引导 13

附录 A：错误码速查表 15

附录 B：测试记录表模板 16

第一章 测试概述

本文档是《造像类文物三维建模自动化系统（路线 E）》的**开发阶段用户实机测试操作指南**，用于指导测试人员（含非专业算法人员）独立完成系统功能验证。系统已完成 **M1~M6 主线全链路**与**可选模块 X1/X2** 的开发，全量 234 项自动化测试通过；本指南在自动化测试之上，给出人工实机测试的完整操作流程、预期结果与判定标准。

> **指南使用说明**：① 按第二章准备环境 → ② 按第三章逐模块执行操作并记录结果 → ③ 用第四章判定标准核对 → ④ 遇问题查第五章应对建议 → ⑤ 附录 B 提供测试记录表模板。全程预计耗时 60~120 分钟。

第二章 测试环境准备

2.1 硬件与软件要求

项目	要求	说明
操作系统	Windows 10/11 64 位	开发与实机测试均基于 Windows
内存	≥ 8 GB（建议 16 GB）	M3 泊松重建与 M5 质检采样为内存敏感环节
磁盘	≥ 10 GB 可用空间	含点云/网格/纹理/报告中间产物
Python	3.12（venv 已配置）	项目使用 .venv_lab 虚拟环境
Python 包	见 requirements.txt	open3d / numpy / py4dgeo / colour-science 等

2.2 环境检查步骤

在命令行（Git Bash 或 PowerShell）中依次执行以下检查，全部通过后方可开始测试：

```
# ① 进入项目目录
cd E:\h3d_lablab

# ② 验证 Python 环境与版本
.venv_lab/Scripts/python.exe -m h3d.cli --version
# 预期输出：h3d 0.1.0

# ③ 验证关键依赖可导入
.venv_lab/Scripts/python.exe -c "import open3d, numpy, py4dgeo, colour; print('deps OK')"
# 预期输出：deps OK

# ④ 运行自动化自检（约 40 秒）
.venv_lab/Scripts/python.exe -m pytest -q
# 预期输出：234 passed（全部通过）
```

2.3 测试数据准备

系统以多站扫描点云（PLY/PCD/XYZ）为输入。测试可用两类数据：

数据来源	组织方式	要求
自带示例数据（推荐初测）	data/raw/<item_id>/scan_sNN.ply	同一造像 ≥ 2 站、有重叠、带颜色
真实扫描数据	00_raw/<item_id>/ 下放置各站文件	文件格式在 project.formats 列表内

目录规范（设计书 §8.3）：00_raw 原始 → 10_clean 清理 → 20_registered 配准 → 30_dense 网格 → 50_tex 带纹理 → 60_qc 质检中间量 → 70_report 质检报告 → 80_export 交付模型 → 90_meta 元数据/清单 → 99_runs 运行归档。用 **init 命令**一键创建骨架：

```
.venv_lab/Scripts/python.exe -m h3d.cli init -p D:\test_project
# 预期输出：已初始化项目目录结构：D:\test_project
# 将测试点云复制到 D:\test_project\00_raw\<item_id>\ 下
```

2.4 配置文件准备

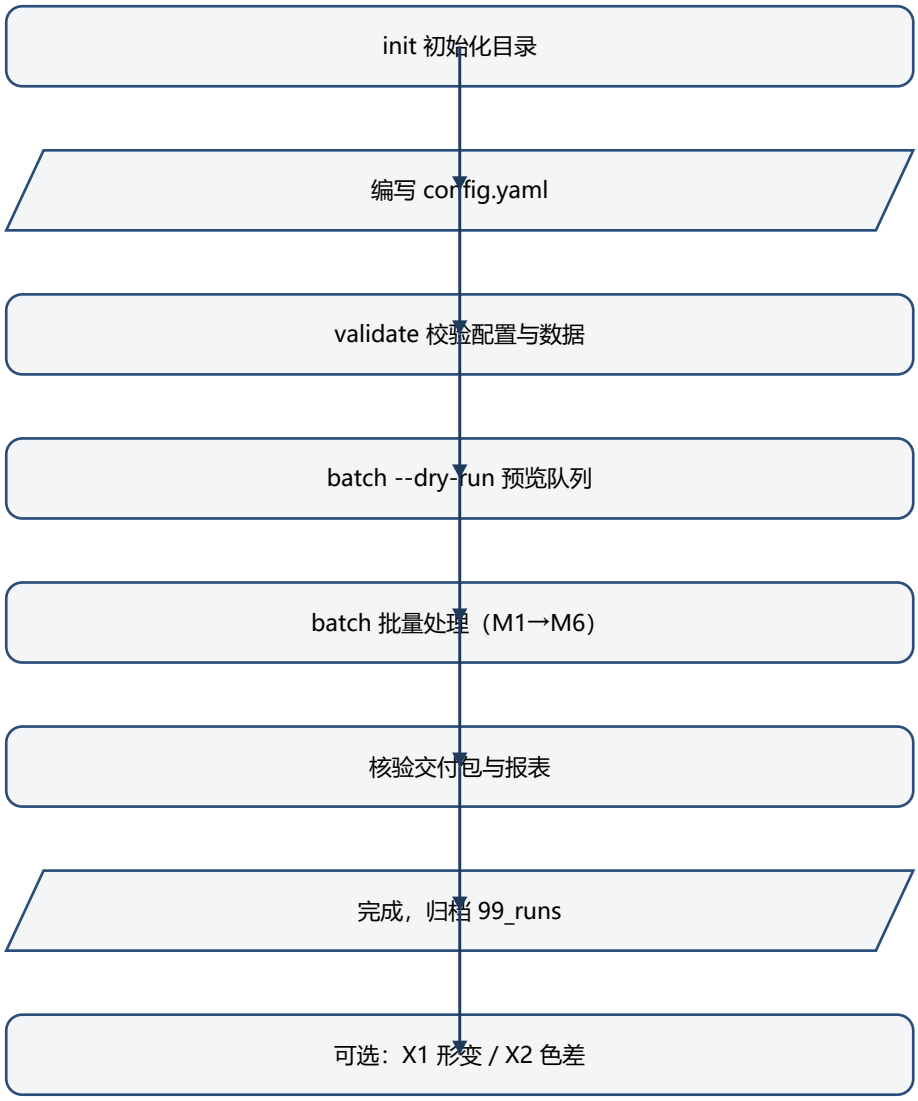
在项目根目录创建 config.yaml（可用 configs/example.yaml 为模板），关键字段：

```
project:
  name: test01        # 项目名
```

```
  raw_dir: D:/test_project/00_raw   # 原始数据目录
  formats: [ply]        # 输入格式
register:
  fpfh_radius: 0.02     # 尺度：点间距小则调小
  ransac_distance: 0.05
mesh:
  poisson_depth: 8      # 8-10（大场景可 6-7）
quality:
  coverage_threshold: 0.80   # 覆盖率质量门
  rmse_threshold_mm: 5.0     # 中误差质量门（WW/T 精细级）
runner:
  manifest: D:/test_project/90_meta/run_manifest.json
report:
  enabled: true
  formats: [csv, png]
  output_dir: D:/test_project/report
```

第三章 各功能模块操作流程

系统按「命令行工具」与「可选分析模块」两类入口操作。主线流程：配置校验 → 批量处理（M1~M6 自动串联）→ 报表与交付包核验 → 可选 X1/X2 分析。整体数据流如下：



实机测试主流程：初始化 → 配置 → 校验 → 预览 → 批量处理 → 交付核验 →（可选 X1/X2）

3.1 配置校验（validate）

目的：确认配置合法、数据完整，避免批量处理中途才发现问题。

```
.venv_lab/Scripts/python.exe -m h3d.cli validate -c config.yaml
# 预期输出示例：
#   配置合法：项目名 = test01
#   数据完整性：发现 2 尊造像，共 4 个采集站点文件
#     - rock001: 2 站
#     - rock002: 2 站
```

3.2 队列预览（batch --dry-run）

目的：仅预览将处理的造像与站点，不执行计算（快速检查）。

```
.venv_lab/Scripts/python.exe -m h3d.cli batch -c config.yaml --dry-run
# 预期输出：[dry-run] 预览队列：2 尊造像
```

3.3 批量处理（batch，核心功能）

目的：一键执行 M1 预处理 → M2 配准 → M3 网格重建 → M4 纹理 → M5 质检 → M6 发布，无人值守。全程日志输出各阶段完成信息；任一尊失败会**失败隔离**（不阻塞其余尊）。

```
.venv_lab/Scripts/python.exe -m h3d.cli batch -c config.yaml
# 预期日志（节选）：
#   M1 完成：rock001（2 站，3000 -> 2925 点）
#   M2 完成：rock001（1 站配准，末站 fitness=0.450）
#   M3 完成：rock001（2925 点 -> 12238 顶点 / 24282 面）
#   M4 完成：rock001（vertex-color，2 LOD x 3 格式）
#   M5 完成：rock001（coverage=74.4%, rmse=211.22mm -> pass）
#   M6 完成：rock001（3 格式模型 + 元数据快照 + 交付清单）
#   批量处理完成：total=2 done=2 failed=0 skipped=0
#   报表产物 -> .../report/metrics_summary.csv
#   报表产物 -> .../report/stage_timings.png
```

3.4 报表重生成（report）

目的：不重跑计算，仅基于既有 manifest 重新生成报表（调参后对比用）。

```
.venv_lab/Scripts/python.exe -m h3d.cli report -c config.yaml
# 预期输出：报表产物 -> ...（重新生成 csv/png）
```

3.5 交付包核验（M6 产物）

目的：确认每尊交付包完整（模型 + 质检报告 + 元数据 + 交付清单）。

```
# 交付包结构检查（Windows 资源管理器或命令行）：
dir D:\test_project\80_export\rock001
# 预期：rock001.ply / rock001.obj / rock001.glb / accuracy.csv
dir D:\test_project\90_meta
# 预期：rock001_meta.json / delivery_manifest.json / run_manifest.json

# 交付清单内容核验：
.venv_lab/Scripts/python.exe -c "import json; d=json.load(
  open('D:/test_project/90_meta/delivery_manifest.json',encoding='utf-8'));
  print(d['n_items'], d['n_published'])"
```

3.6 可选模块 X1 M3C2 多时相（独立调用）

目的：对同一造像两期（t1/t2）配准点云做 M3C2 差分，量化风化/堆积变化。X1 不进入主线 stage，以 Python API 独立调用（两期数据自行组织）。

```
from h3d.core import ChangeConfig
from h3d.change import run as change_run
from h3d.io import load_point_cloud

t1 = load_point_cloud('20_registered/t1/rock001.ply')  # 基准时相
t2 = load_point_cloud('20_registered/t2/rock001.ply')  # 监测时相
r = change_run(t1, t2, ChangeConfig(cores=2, change_mm=2.0, grid=[32, 32]))
print(r['metrics'])  # max_change_mm / changed_ratio / n_corepoints
```

3.7 可选模块 X2 CIEDE2000 色差（独立调用）

目的：评估建模纹理与实采色卡（ColorChecker24）的色彩保真度。X2 以 Python API 独立调用，实测色块由测试方从建模纹理中提取。

```
from h3d.core import ColourConfig
from h3d.colour import run as colour_run
from h3d.colour.color_calib import COLORCHECKER24_SRGB

# measured：24 个实测色块 sRGB（0~1），顺序对应色卡
r = colour_run(measured, COLORCHECKER24_SRGB,
               ColourConfig(use_calibration=True, jnd_threshold=2.3))
print(r['mean_de'], r['max_de'], r['jnd_ratio'])  # 均值/最大/超标比例
```

第四章 预期结果与判定标准

4.1 逐命令判定表

测试项	操作	预期结果	判定标准（通过条件）
版本	h3d.cli --version	h3d 0.1.0	版本号输出且无 Traceback
自检	pytest -q	234 passed	全部通过；失败数 = 0
初始化	init -p <dir>	已初始化项目目录结构	10 个标准目录全部创建
校验-合法	validate -c cfg	配置合法 + 发现 N 尊	N 与 00_raw 下造像数一致
校验-非法	validate -c bad_cfg	配置错误：[E001] ...	退出码 1，错误信息明确
预览	batch --dry-run	[dry-run] 预览队列：N 尊	N 正确且未执行任何计算
批量	batch -c cfg	M1~M6 全部完成	done=N failed=0，阶段日志齐全
报表	report -c cfg	报表产物 -> csv/png	metrics_summary.csv 可打开
交付包	核验 80_export	每尊 3 格式 + accuracy.csv	文件齐全、模型可被外部软件打开
元数据	核验 90_meta	meta + delivery_manifest + manifest	JSON 可解析、字段完整
续跑	重跑 batch	skipped=N（产物在）	不重复计算，秒级完成
X1	change_run 脚本	metrics 输出	n_corepoints>0，无崩溃
X2	colour_run 脚本	mean_de 输出	mean_de 为有限值，24 色块

4.2 质量门判定标准（M5）

指标	阈值（默认）	判定	依据
覆盖率 coverage	≥ 0.80	低于阈值 -> E500 隔离	设计书 §6.3
中误差 rmse	≤ 5.0 mm	参考足够密时判门	WW/T 0115/0117 精细级
边界边比例	报告项	参考值 ≤ 5%	孔洞率参考

4.3 失败隔离判定

批量处理中某尊某阶段失败（如配准失败 / 质检不达标 / 坏文件）时，该尊被**隔离**，其余尊照常处理。判定：日志出现「失败隔离：item=... stage=... -> ...（继续下一尊）」；report/failures.csv 记录 5 列（文物编号 / 阶段 / 异常类型 / 异常信息 / 时间）；summary 的 failed 计数 = 隔离尊数。该尊不进入交付包（设计书 §6.3）。

第五章 常见问题与应对建议

现象	可能原因	应对建议
pytest 报 234 未通过	依赖版本不符 / 环境未正确激活	重跑 .venv_lab 环境；检查 requirements.txt
validate 报 [E001] 字段越界	配置阈值超出合法域（如覆盖率 >1）	按错误信息修正 config.yaml 对应字段
validate 发现 0 尊	00_raw 目录结构不对 / formats 不匹配	检查 <raw_dir>/<item_id>/ 层级与文件扩展名
batch 报「失败隔离 M2 RegistrationError」	两站重叠不足或配准参数不适配	增大 fpfh_radius / ransac_distance；提高数据重叠度
M5 报 QualityGateError coverage 低	覆盖率未达阈值（点云稀疏/阈值过严）	检查 coverage_k_mps（默认 3.0）与 coverage_threshold；补扫缺失面
重跑全部 skipped=0 且未计算	产物齐全（resume 生效）	属正常：断点续跑跳过已完成阶段；删除产物可强制重跑
删除产物后重跑仍不重算	旧版 resume 只查 manifest（已修复）	确认使用最新代码（v0.1.0+）；删除产物后 batch 会级联重跑
损坏/空点云文件	扫描文件损坏	系统自动隔离该尊并在 failures.csv 记录 [E100]；修复文件后重跑
M4 输出无颜色（colored=False）	扫描点云不带 RGB	正常降级：输出几何模型并标注；带色扫描可获得顶点色纹理
路径含中文/空格	部分工具链兼容性	建议项目目录使用纯 ASCII 路径（系统 io 层已做 UTF-8 处理）
X1/X2 无 py4dgeo/colour	依赖未安装	pip install py4dgeo colour-science（requirements 已列）

通用排查顺序：① 看失败隔离日志（哪尊哪阶段）→ ② 查 report/failures.csv 的异常类型与信息 → ③ 对照错误码表（附录 A）→ ④ 修复后重跑（断点续跑自动跳过已完成阶段）。任何情况下都不应出现进程「突然中断」——系统已统一异常兜底，所有失败都以隔离方式记录。

第六章 后续持续开发方向

6.1 后续可迭代的功能与优化方向

方向	内容	优先级
UV 图集纹理	texture.paint_mode='uv' 实装：xatlas 展开 + 纹理图烘焙（M5 overlap 指标启用）	P1
X1/X2 接入编排	change/colour 条件注册为可选 stage（cfg 开关启用即入队）	P1
并发调度	runner.workers 进程池并行处理独立造像（处理 Open3D 子进程初始化）	P2
HTML 报表	report.formats 增加 html（jinja2 + matplotlib SVG 内嵌）	P2
CloudCompare 交叉校验	change.validation 实装 CC CLI M3C2 完整调用	P2
大场景优化	M3 千万级点云分块重建 / 流式降采样	P3
多时相监测接口	X1 输出对接监测平台（变化热力 + 统计 API）	P3

6.2 面向未来的发展目标与扩展路线

近期（0.2.x）：UV 纹理 + X1/X2 编排化 + 并发调度，达到「实验室可用」；中期（0.3.x~0.5.x）：HTML 报表 + GUI 原型 + 多语种，达到「业务试用」；远期（1.0）：GUI 完备化 + 品牌发布 + 云端服务（对接监测平台 / 色彩管理流程，设计书 §6 旁系）。扩展路线图：



版本演进路线：命令行 → 工程化 → GUI 化 → 品牌化发布

6.3 GUI 开发方案与界面设计思路

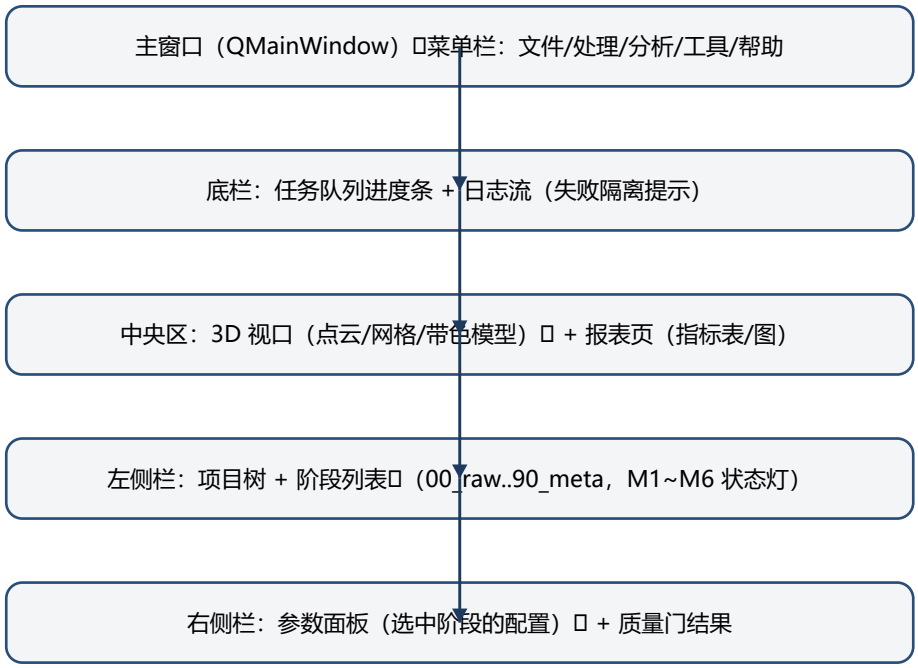
GUI 方案选型：**PySide6（LGPL，与现有纯 Python 栈一致）**为主，Qt3D/OpenGL 3D 预览；备选 DearPyGui（MIT，ImGui 风格，开发最快）。设计思路：把 CLI 的四个命令映射为「向导式 + 仪表盘式」双模式：① 新建项目向导（目录/数据/参数分步）；② 主工作台（项目树 + 3D 视口 + 队列进度 + 报表页）。界面语言默认中文（文保单位用户）。

6.4 完整的软件化改造路径

软件化改造 = 命令行内核（已就绪）+ GUI 外壳 + 打包分发：① 内核保持纯函数式 API（core/io/runner 与 GUI 解耦，设计书 §6.4 一致性）；② GUI 层只做「参数编辑 → 调用内核 → 展示结果」；③ 打包用 PyInstaller（--onefile）生成 Windows 可执行文件，附 venv 依赖；④ 用 PySide6 的 Qt 资源系统（.qrc）嵌入图标与样式表。

6.5 完备 GUI 窗口的架构设计

建议的单窗口多面板架构（主窗口 + 左右侧栏 + 底栏）：



GUI 单窗口五区架构：菜单/项目树/3D 视口+报表/参数面板/队列进度日志

组件	技术选型	要点
3D 视口	Qt3D / open3d.visualization 嵌入	点云/网格/带色模型切换显示
项目树	QTreeView + 自定义 Model	目录结构 + 阶段状态（done/fail/skip）
参数面板	QFormLayout + 校验器	与 Config 描述符联动（越界即红框）
报表页	QWebEngineView / matplotlib 嵌入	HTML 报表直接内嵌
队列底栏	QProgressBar + QPlainTextEdit	进度 + 结构化日志流

6.6 软件图标化、品牌化与发布打包引导

① **图标化**：设计 256x256 应用图标（建议：造像轮廓 + 点云网格线稿，蓝绿渐变，呼应「文保数字化」主题）；用 SVG 源文件导出多尺寸（16/32/48/256），Windows 用 .ico；PySide6 用 QIcon 嵌入。② **品牌化**：统一应用名（建议中文名 + 英文代号）、版本号语义化（0.1.0）、splash 屏与关于页含版权/许可（MIT/BSD 声明）。③ **发布打包**：PyInstaller --onefile --windowed，附 configs/example.yaml 与 README；用 CI （GitHub Actions）自动构建 Windows 安装包；发布到内网仓库或网盘，随版本附「变更日志 + 测试报告」。

路线图优先级建议：先做 6.1 的 P1 项（UV 纹理 + X1/X2 编排化），再启动 6.3 的 GUI 原型（可并行）；GUI 用 PySide6 从「新建项目向导 + 批量处理仪表盘」两个窗口起步，逐步补全 6.5 五区架构；打包发布在 GUI 功能冻结后一次性完成。

附录 A：错误码速查表

错误码	含义	典型场景
E001	配置错误	字段越界 / 配置缺失 / 非法取值
E100	输入数据错误	空点云 / 文件无法读取 / MPS 估计失败
E200	配准失败	两站重叠不足 / 特征匹配失败
E500	质检不达标	覆盖率或中误差未过质量门（隔离）
E601	导出失败	格式不支持 / 写盘失败
E900	报告写入失败	目标目录不可写

附录 B：测试记录表模板

用例编号	模块	操作	预期	实际	结论(通过/失败)	备注
T-01	启动	h3d --version	h3d 0.1.0			
T-02	自检	pytest -q	234 passed			
T-03	初始化	init -p <dir>	10 目录创建			
T-04	校验	validate -c cfg	配置合法+N 尊			
T-05	预览	batch --dry-run	队列 N 尊			
T-06	批量	batch -c cfg	M1~M6 done			
T-07	报表	report -c cfg	csv/png 生成			
T-08	交付包	核验 80_export/90_meta	模型+报告+元数据			
T-09	续跑	重跑 batch	skipped=N			
T-10	异常	坏文件/低覆盖率	隔离+failures.csv			
T-11	X1	change_run	形变统计输出			
T-12	X2	colour_run	色差统计输出			